UNIVERSITY REGISTRATION DATABASE

Full-Stack Software Development — Lab Sheet 06

Student name	_____
Student ID	_____
Course code	COSC 32133 / BECS 32263
Course name	Full-Stack Software Development
Academic year	2024/2025 — Semester II
Submission date	_____
GitHub repository URL	_____

Submission draft — complete the blank identity fields and insert genuine screenshots from your own MySQL and Spring Boot execution before submitting.

1. Introduction

This lab redesigns a university course-registration spreadsheet as a relational database normalized to Third Normal Form (3NF). It then creates the schema in MySQL and maps the tables to Java entities using Spring Data JPA. The application exposes read-only REST endpoints to demonstrate repository queries.

2. First Normal Form (1NF)

The original spreadsheet stores multiple course registrations in repeating groups within a student row. To reach 1NF, each student/course combination is represented as a separate row and each cell contains one atomic value. The composite primary key is (student_id, course_code).

Student	Student name	Course	Course title	Credits	Lecturer	Department	Grade
S001	Kamal Silva	CS101	Intro to Programming	3	Dr. Perera	Computer Science	A
S001	Kamal Silva	CS102	Database Systems	3	Dr. Fernando	Computer Science	B+
S002	Nimali Perera	CS101	Intro to Programming	3	Dr. Perera	Computer Science	B
S002	Nimali Perera	ST201	Probability Theory	3	Dr. Jayasinghe	Statistics	A-
S003	Saman Fernando	CS102	Database Systems	3	Dr. Fernando	Computer Science	C+
S003	Saman Fernando	ST201	Probability Theory	3	Dr. Jayasinghe	Statistics	B
S004	Dilini Jayawardena	CS201	Web Development	4	Dr. Perera	Computer Science	A

The flat table has redundancy and update, insert and delete anomalies. Lecturer e-mail and department information repeat across registrations. Updating only one repeated row creates inconsistent facts; a course cannot be inserted without a student; deleting the only enrollment for a course also removes the course details.

3. Functional Dependencies

Determinant	Dependent attributes
StudentID	StudentName, StudentEmail
CourseCode	CourseTitle, Credits, LecturerEmail
(StudentID, CourseCode)	Grade
LecturerEmail	LecturerName, DeptCode
DeptCode	DeptName

These dependencies follow the lab scenario assumptions: each course has one lecturer; each lecturer has one e-mail address and belongs to one department; and each department code identifies one department name.

4. Second Normal Form (2NF)

The 1NF key is composite. Student details depend only on StudentID, while course details depend only on CourseCode. Grade depends on the complete student/course pair. To remove partial dependencies, split the data into:

STUDENT_2NF (student_id PK, student_name, student_email)
COURSE_2NF (course_code PK, course_title, credits, lecturer_name, lecturer_email, department_code, department_name)
ENROLLMENT_2NF (student_id PK/FK, course_code PK/FK, grade)

5. Third Normal Form (3NF)

The course-side attributes contain transitive dependencies: CourseCode → LecturerEmail → LecturerName, DeptCode and DeptName; and DeptCode → DeptName. Separate lecturer and department facts into their own tables. The final five tables are:

DEPARTMENT (id PK, code UNIQUE, name)
LECTURER (id PK, name, email UNIQUE, department_id FK)
COURSE (id PK, code UNIQUE, title, credits, lecturer_id FK)
STUDENT (id PK, student_code UNIQUE, full_name, email UNIQUE)
ENROLLMENT (id PK, student_id FK, course_id FK, grade; UNIQUE(student_id, course_id))

Each non-key attribute depends on the key, the whole key and nothing but the key. The final design avoids storing lecturer and department facts repeatedly.

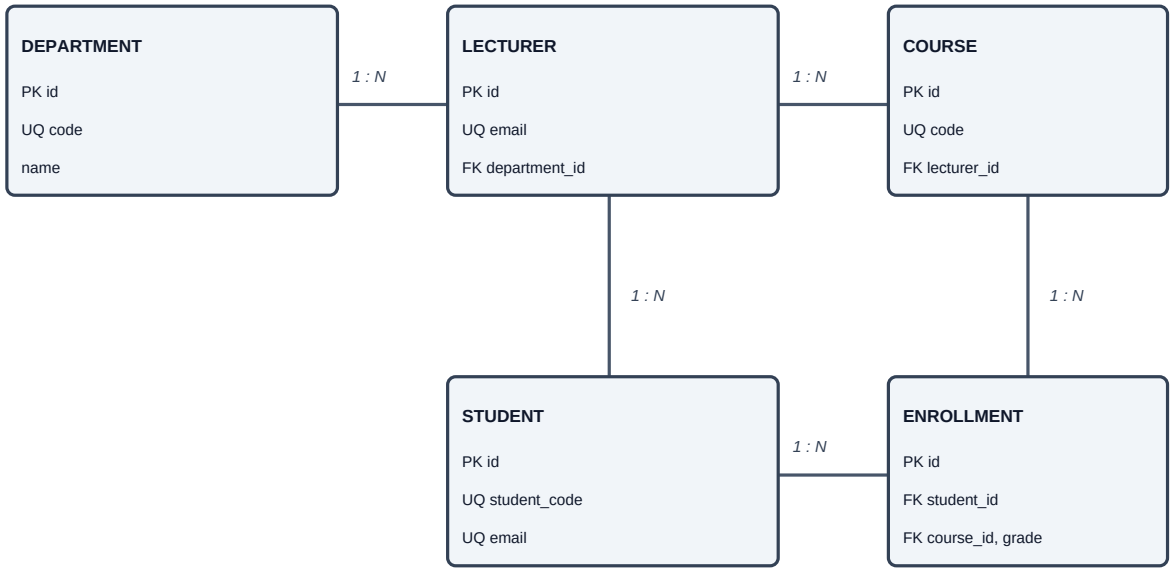


Figure 1. Logical ER diagram of the five-table 3NF design. Each parent record can be related to many child records.

6. MySQL Schema and Constraints

The schema is created in dependency order: department, lecturer, course, student and enrollment. Primary keys identify rows; UNIQUE constraints protect natural identifiers and e-mail addresses; NOT NULL enforces required values; CHECK restricts credits to 1–6; and foreign keys enforce referential integrity. The SQL scripts are included in the accompanying project ZIP.

7. ORM and JPA Mapping

JPA maps each relational table to an entity class: Department, Lecturer, Course, Student and Enrollment. Lecturer has a @ManyToOne relationship to Department; Course has a @ManyToOne relationship to Lecturer; Enrollment has @ManyToOne relationships to Student and Course. Department exposes its lecturers through @OneToMany(mappedBy="department"). Hibernate is configured with ddl-auto=validate so it checks the existing database schema without modifying it.

8. Repository Queries and REST API

The project includes Spring Data JPA repositories with derived query methods such as findByStudentCode, findByCreditsGreaterThanEqual, findByLecturer_Department_Name and findByGrade. DepartmentRepository uses JPQL with LEFT JOIN FETCH to load a department and its lecturers together. EnrollmentRepository uses a JPQL GROUP BY query to count enrollments per course. The read-only controller exposes endpoints under /api for students, courses, departments, enrollments and course-count reports.

9. Expected Query Results

Endpoint	Expected result

/api/students	4 students, sorted by name; Dilini first
/api/students?name=sil	Kamal Silva
/api/students/S001	Kamal Silva
/api/students/S001/enrollments	CS101 (A) and CS102 (B+)
/api/courses?minCredits=4	CS201 only
/api/courses?department=Computer Science	CS101, CS102 and CS201
/api/departments/D01/lecturers	Dr. Perera and Dr. Fernando
/api/enrollments?grade=A	S001/CS101 and S004/CS201
/api/reports/course-counts	CS101: 2, CS102: 2, ST201: 2, CS201: 1

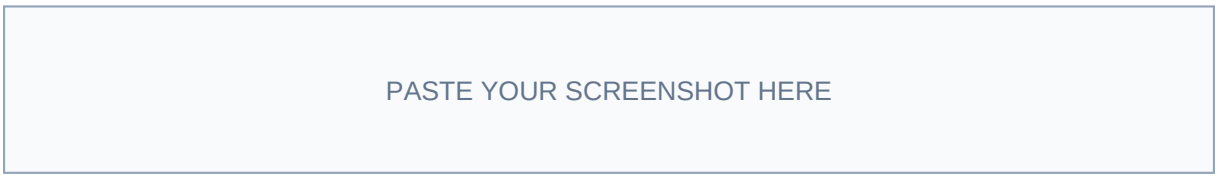
For the department-filtered course endpoint, the generated SQL joins course to lecturer and lecturer to department. For grade-filtered enrollments, additional SELECT statements may occur because the related Student and Course entities are loaded, followed by Lecturer and Department relationships. JOIN FETCH reduces this for the department-and-lecturers query by fetching the related collection in one SQL statement.

10. Evidence to Insert Before Submission

The following evidence must be captured from your own computer. Replace each placeholder with a genuine screenshot and add a short caption. Do not leave this section empty in the final submission.

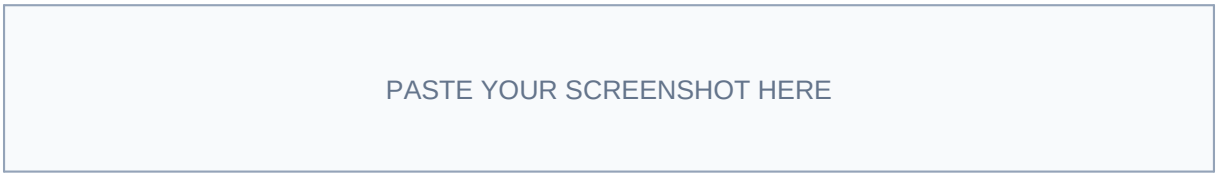
Screenshot 1 — MySQL server

Show phpMyAdmin and the SHOW DATABASES result.



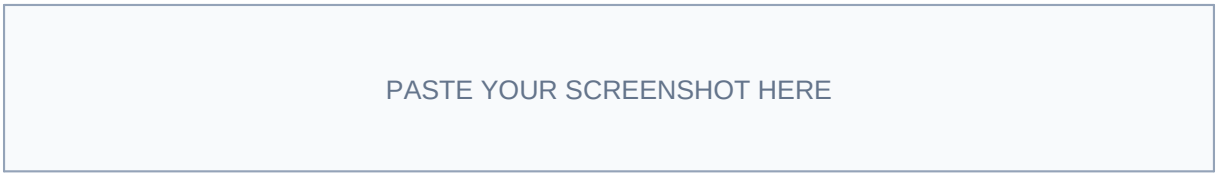
Screenshot 2 — 1NF and anomalies

Show the 7-row registration_flat table and actual results for redundancy/update, insert and delete anomaly experiments.



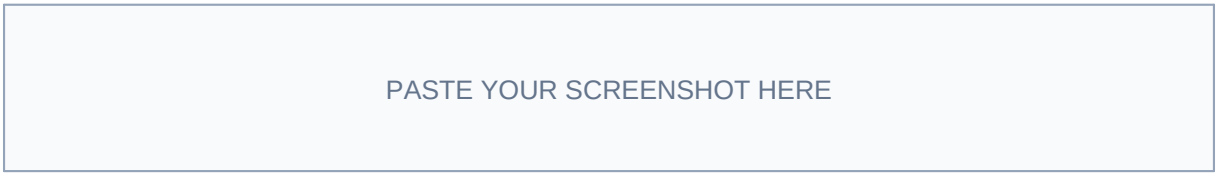
Screenshot 3 — 3NF schema

Show SHOW TABLES, DESCRIBE student, DESCRIBE enrollment and sample table data.



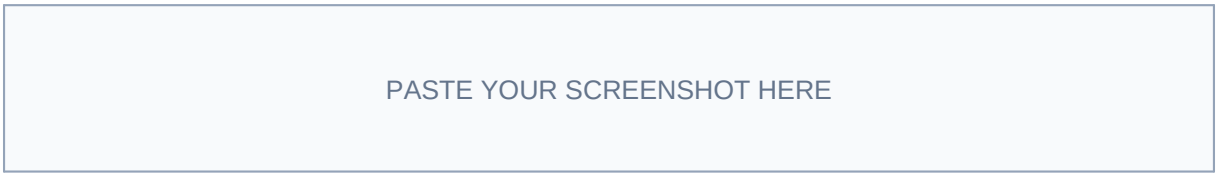
Screenshot 4 — ER diagram

Show the five tables and relationships in phpMyAdmin Designer or MySQL Workbench.



Screenshot 5 — Fixed-schema tests

Show the actual error messages from each of the eight failed statements and the temporary ALTER TABLE add/drop test.



Screenshot 6 — Spring Boot configuration

Show project dependencies and application.properties with ddl-auto=validate. Hide any real database password.

PASTE YOUR SCREENSHOT HERE

Screenshot 7 — JPA validation

Show successful application startup and the temporary missing-column validation failure, then the restored successful startup.

PASTE YOUR SCREENSHOT HERE

Screenshot 8 — API tests

Show browser/Postman responses for the listed API endpoints.

PASTE YOUR SCREENSHOT HERE

Screenshot 9 — SQL console

Show generated SQL for the department filter, grade filter and JOIN FETCH query.

PASTE YOUR SCREENSHOT HERE

Screenshot 10 — GitHub

Show the repository, feature branch, merged pull request and final main branch.

PASTE YOUR SCREENSHOT HERE

11. Conclusion

The lab demonstrates how normalization reduces redundancy and prevents update, insert and delete anomalies. The resulting 3NF schema enforces data integrity through keys and constraints. JPA maps the database tables to Java entities, while Spring Data repositories and REST endpoints provide a way to retrieve and inspect the registration data.

Final check: complete the student details, add all genuine screenshots, verify the code runs on your machine, and replace the GitHub URL with your actual repository URL.